Alien Attack
Activité préparatoire N°1

Préparation au projet

1 Étape N°1 : Un carré errant

1.1 Modifier un programme

Vous disposez du programme ci-dessous qui permet de déplacer un carré violet dans une fenêtre sur un fond noir.

Objectifs Etape N°1.
Vous devez modifier ce programme pour atteindre les objectifs ci-dessous, faites valider par le professeur chaque fois qu'un objectif est atteint :

- ☐ Le carré doit être Rouge
- ☐ Le carré doit avoir une taille de 4×4, une fois l'objectif validé, remettre un carré 8×8.
- ☐ Le carré doit pouvoir se déplacer dans toute la fenêtre, pas seulement à droite et à gauche.
- ☐ Le carré ne doit pas pouvoir sortir de la fenêtre.

```python
import pyxel

pyxel.init(128, 128, title="Carre errant")

carre_x = 60
carre_y = 60

def carre_deplacement(x, y):
        if pyxel.btn(pyxel.KEY_RIGHT):
                x = x + 1
        if pyxel.btn(pyxel.KEY_LEFT):
                x = x - 1
        return x, y

def update():
        global carre_x, carre_y
        carre_x, carre_y = carre_deplacement(carre_x, carre_y)

def draw():
        pyxel.cls(0)
        pyxel.rect(carre_x, carre_y, 8, 8, 1)

pyxel.run(update, draw)
```

CC BY NC SA

1.2 Compléments : Variables locales et variables locales en python

En Python, nous pouvons déclarer des variables n'importe où dans notre script : au début du script, à l'intérieur de boucles, au sein de nos fonctions, etc.

L'endroit où on définit une variable dans le script va déterminer l'endroit où la variable va être accessible c'est-à-dire utilisable.

Le terme de **portée des variables** sert à désigner les différents espaces dans le script dans lesquels une variable est accessible c'est-à-dire utilisable. En Python, une variable peut avoir une **portée locale** ou une **portée globale**.

1.2.1 Les variables locales

Déf. **Les variables définies dans une fonction sont appelées variables locales.** Elles ne peuvent être utilisées que localement c'est-à-dire qu'à l'intérieur de la fonction qui les a définies. Tenter d'appeler une variable locale depuis l'extérieur de la fonction qui l'a définie provoquera une erreur.

```
def fonction() :
        x = 4
        return x

>>> fonction()
4
>>> x
NameError: name 'x' is not defined
```

Cela est dû au fait que chaque fois qu'une fonction est appelée, Python réserve pour elle (dans la mémoire de l'ordinateur) un nouvel espace de noms (c'est-à-dire une sorte de dossier virtuel). Les contenus des variables locales sont stockés dans cet espace de noms qui est inaccessible depuis l'extérieur de la fonction.

Cet espace de noms est automatiquement détruit dès que la fonction a terminé son travail, ce qui fait que les valeurs des variables sont réinitialisées à chaque nouvel appel de fonction.

1.2.2 Les variables globales

Déf. **Les variables définies dans l'espace global du script, c'est-à-dire en dehors de toute fonction sont appelées des variables globales.** Ces variables sont accessibles (= utilisables) à travers l'ensemble du script et accessibles en **lecture seulement** à l'intérieur des fonctions utilisées dans ce script.

```
# x est une variable globale
x = 4

def fonction() :
        return x

>>> fonction()
4
>>> x
4
```

```
# x est une variable globale
x = 4
# Mais c'est aussi une varible locale
def fonction(x) :
        x = x + 4
        return x
>>> fonction(x)
8
>>> x
4
```

Attention

Une fonction va pouvoir utiliser la valeur d'une variable définie globalement mais ne va pas pouvoir modifier sa valeur c'est-à-dire la redéfinir. En effet, toute variable définie dans une fonction est par définition locale ce qui fait que si on essaie de redéfinir une variable globale à l'intérieur d'une fonction on ne fera que créer une autre variable de même nom que la variable globale qu'on souhaite redéfinir mais qui sera locale et bien distincte de cette dernière.

1.2.3 Modifier une variable globale depuis une fonction

Dans certaines situations, il serait utile de pouvoir modifier la valeur d'une variable globale depuis une fonction, notamment dans le cas où une fonction se sert d'une variable globale et la manipule.

Cela est possible en Python. Pour faire cela, il suffit d'utiliser le mot clef `global` devant le nom d'une variable globale utilisée localement afin d'indiquer à Python qu'on souhaite bien modifier le contenu de la variable globale et non pas créer une variable locale de même nom.

```
x = 4

def fonction() :
        global x
        x = x + 6
        return x

>>> fonction()
10
>>> x
10
```

```
x = 4

def fonction() :
        x = x + 6
        return x

>>> fonction()
UnboundLocalError: local variable 'x' \
referenced before assignment
>>> x
4
```

2 Étape n°2 : Ajouter des tirs

On souhaite maintenant que notre carré puisse tirer des salves laser sur les ennemis qui ne vont pas tarder à arriver. Pour cela on propose de rajouter deux fonctions :

```
1
2 def tirs_creation(x, y, tirs_liste):
3         # btnr pour eviter les tirs multiples
4         if pyxel.btnr(pyxel.KEY_SPACE):
5                 tirs_liste.append([x, y])
6         return tirs_liste
7
8
9 def tirs_deplacement(tirs_liste):
10        for tir in tirs_liste:
11                tir[0] -= 1
12                if  tir[0]<-8:
13                        tirs_liste.remove(tir)
14        return tirs_liste
```

Objectifs Étape N°2.

Vous devez modifier ce programme pour atteindre les objectifs ci-dessous, faites valider par le professeur chaque fois qu'un objectif est atteint :

- ☐ Insérer dans votre code les fonctions ci-dessus.
- ☐ Déclarer une liste vide `tirs_liste` en début de script.
- ☐ Insérer au bon endroit la portion de code ci dessous qui dessine les tirs, en remplaçant les pointillés par le code adéquat :

```
for .... in tirs_liste:
        pyxel.rect(tir[..], tir[..], 1, 4, 10)
```

- ☐ Il faut maintenant appeler correctement les fonctions `tirs_creation` et `tirs_deplacement` dans la fonction `update`. Compléter le corps de cette dernière en modifiant la variable `tirs_liste`
- ☐ Les tirs se déplacent de la droite vers la gauche, modifiez le code pour que les tirs se déplacent du bas vers le haut, en étant éliminés de la liste lorsqu'ils quittent l'écran.
- ☐ Les tirs devraient partir du centre du carré rouge,ce n'est pas le cas. Modifiez le code pour que les tirs partent correctement du centre du carré rouge.

3 Étape N°3 : Ajouter les ennemis

L'objectif de cette étape est de maintenant ajouter des ennemis, qui vont arriver par le haut de la fenêtre et descendre verticalement. **On veut qu'un ennemi arrive toutes les secondes.**

```
def ennemis_creation(ennemis_liste):
        if (.......................):
                ennemis_liste.append([random.randint(0, 120), 15])
        return ennemis_liste


def ennemis_deplacement(ennemis_liste):
        for ennemi in ennemis_liste:
                ennemi[1] += ....
                        if  ennemi[1]>120:
                                ennemis_liste.remove(ennemi)
        return ennemis_liste
```

Attention

Le module pyxel dispose d'une variable `pyxel.frame_count` qui compte le nombre de *frame*, le nombre d'images, depuis le lancement du programme. Par défaut, pyxel génère 30 frames par seconde.
Un ennemi doit donc apparaitre tous les multiples de 30 frames.

CC BY NC SA

Objectifs Étape N°3.

Vous devez modifier ce programme pour atteindre les objectifs ci-dessous, faites valider par le professeur chaque fois qu'un objectif est atteint :

- ☐ Insérer dans votre code les fonctions `ennemis_creation(ennemis_liste)` et `ennemis_deplacement(ennemis_liste)` ci-dessus en les complétant correctement.
- ☐ Déclarer une liste vide `ennemis_liste` en début de script,au début du jeu il n'y a pas d'ennemis !
- ☐ Insérer au bon endroit la portion de code ci dessous qui dessine les ennemis présents dans la liste `ennemis_liste`, en remplaçant les pointillés par le code adéquat :

```
for ....... in ..........:
        pyxel.rect(............, ............, 8, 8, 10)
```

- ☐ Il faut maintenant appeler correctement les fonctions `ennemis_creation(ennemis_liste)` et `ennemis_deplacement(ennemis_liste)` dans la fonction `update` pour mettre à jour la liste `ennemis_liste`. Compléter le corps de la fonction `update`.

4 Étape 4 : Gérer les collisions

4.1 Éliminer les ennemis

Dans un premier temps il va s'agir de supprimer les ennemis touchés par des tirs. Pour cela on propose une fonction :

```
def ennemis_suppression():
  for ennemi in ennemis_liste:
    for tir in tirs_liste:
      if ennemi[0] ... tir[0]+1 and ennemi[0]+8 ... tir[0] and ennemi[1]+8 ... tir[1]:
        ennemis_liste.remove(ennemi)
        tirs_liste.remove(tir)
```

Objectifs Étape N°4a.

- ☐ De quels types sont les variables `ennemi` et `tir` ?
- ☐ Compléter le code de la fonction `ennemis_suppression()` et l'ajouter à votre code. On réfléchira bien sur les conditions de collision entre un tir et un ennemi
- ☐ Cette fonction est un peu particulière, elle fonctionne par *'effet de bord'*. Après avoir lu le paragraphe ci-dessous *Compléments : les effets de bord*, expliquer pourquoi cette fonction permet de supprimer les tirs qui touchent les ennemis et les ennemis touchés.
- ☐ Insérer un appel à cette fonction dans le corps de la fonction `update`

CC BY NC SA

4.2 Compléments : Les effets de bord

Déf.

En informatique, une fonction est dite à effet de bord (traduction de l'anglais side effect) si elle modifie un état en dehors de son environnement local, autre que renvoyer une valeur.

Par exemple, les fonctions qui modifient une variable globale ou un argument mutable passé par référence sur un pointeur ou les fonctions qui effectuent des opérations d'entrées-sorties.

Une fonction est donc à effet de bord si elle modifie une variable en dehors de son environnement local. En Python, cela se rencontre souvent avec les structures mutables comme les tableaux (de type list) ou les dictionnaires (de type dict).

```
# échange de 2 élèments d'une liste
# avec effet de bord

def echange(liste, indice1, indice2):
    liste[indice1] = temp
    liste[indice1] = liste[indice2]
    liste[indice2] = temp
```

```
# échange de 2 élèments d'une liste
# sans effet de bord

def echanger(liste, indice1, indice2):
    nvListe = [element for element in liste]
    nvListe[indice1] = temp
    nvListe[indice1] = nvListe[indice2]
    nvListe[indice2] = temp
   return nvListe
```

Attention : Il faut être prudent avec les effets de bord !

- Difficulté à déboguer :
  Si une fonction modifie des variables en dehors de son propre contexte, il devient difficile de comprendre pourquoi ou comment un état particulier a changé. Cela peut introduire des bugs subtils difficiles à détecter.
- Manque de lisibilité :
  Un code avec des effets de bord est moins intuitif, car il n'est pas évident de prédire tous les impacts d'une fonction. Cela complique la lecture et la compréhension du flux du programme.
- Réutilisabilité limitée :
  Une fonction qui a des effets de bord dépend souvent fortement de son contexte d'exécution, ce qui limite sa réutilisation dans d'autres parties du code ou projets.
- Dépendances cachées :
  Une fonction avec des effets de bord peut dépendre d'une variable globale ou d'un état externe, rendant le comportement de cette fonction dépendant de l'environnement dans lequel elle est utilisée.

4.3 Destruction du vaisseau

Nous allons maintenant nous intéresser à la destruction du vaisseau quand il est percuté par un ennemi. Nous devons donc ajouter plusieurs choses à notre code. Tout d'abord, une variable `vies` qui comptera le nombre de vies restantes. Il conviendra donc de la déclarer comme variable globale dans la focntion `update`

```
# En début de code
vies = 3
```

Il nous maintenant écrire une fonction qui va tester la collision entre notre carré et les ennemis présents dans l'aire de jeu. Cette fonction diminuera le nombre de vies de notre carré. Elle modifie par *effet de bord* la liste `ennemi_liste` et renvoie le nouveau nombre de vie.

CC BY NC SA

```
def vaisseau_suppression(vies):
  for ennemi in ennemis_liste:
    if ennemi[0] <= carre_x... and ennemi[1] <= carre_y... and \
    ennemi[0]... >= carre_x and ennemi[1]... >= carre_y:
      ennemis_liste.remove(ennemi)
      vies -= 1
  return vies
```

Objectifs Étape N°4b.

- ☐ Compléter la fonction `vaisseau_suppression` et l'intégrer à votre code.
- ☐ Appeler cette fonction dans la fonction `update`, en affectant sa valeur de retour à la variable`vies`, déclarée comme *globale*.
- ☐ Dans la fonction `draw`, on peut maintenant afficher le nombre de vies :

```
pyxel.text(5,5, 'VIES:'+ str(vies), 7)
```

- ☐ Dans cette focntin `draw`, on peut maintenant gérer la fin du jeu. Le principe est simple, si le nombre de vies est plus grand que 0, on affiche le carré, les tirs et les ennemis, sinon on affiche un joli **Game Over**, au centre de l'écran. Implémentez cette fonctionnalité

CC BY NC SA